- RAG Learning

1.2 Memory and Storage

OCR GCSE Computer Science (J277) — Paper 1

1.2.1 Primary Storage — RAM, ROM and Cache

Primary storage (also called main memory) is memory that is directly accessible by the CPU via the system bus. It includes RAM, ROM, and cache. Primary storage is fast but limited in capacity compared to secondary storage. The CPU cannot process data directly from secondary storage — everything must first be loaded into primary storage.

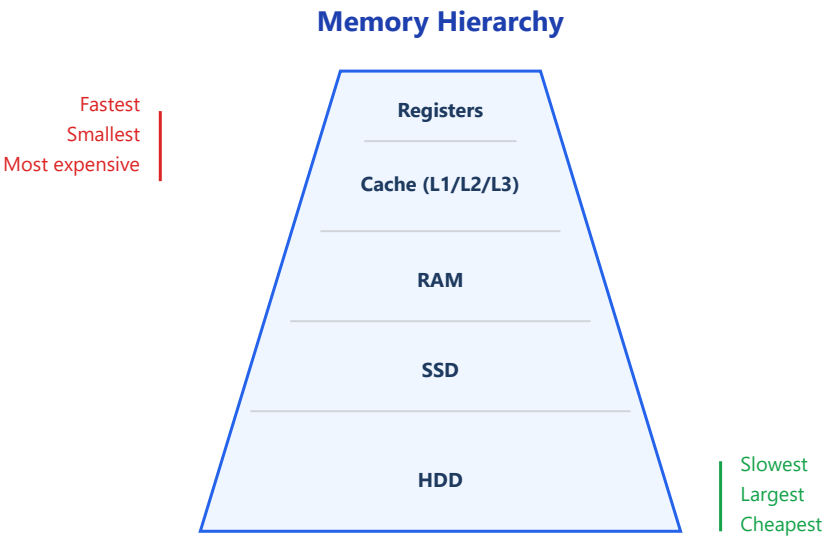
Why Do We Need Primary Storage?

Secondary storage (hard drives, SSDs) is far too slow for the CPU to work with directly. The CPU operates in nanoseconds, while even an SSD takes microseconds to respond — a thousand times slower. RAM bridges this gap by holding the programs and data the CPU is currently using. When you open a program, it is copied from secondary storage into RAM, where the CPU can access it millions of times faster.

The Memory Hierarchy

Computer storage is organised in a hierarchy from fastest/smallest/most expensive at the top to slowest/largest/cheapest at the bottom. Each level acts as a buffer for the level below it:

© RAG Learning · Not for redistribution



1. **Registers** — inside CPU, fastest, a few bytes (e.g., 16 registers of 8 bytes each, ~1 ns access)
2. **L1 Cache** — on CPU die, very fast, 32-64 KB per core (~1 ns access)
3. **L2/L3 Cache** — on CPU die, fast, 256 KB - 32 MB (~3-20 ns)
4. **RAM** — main memory, fast but volatile, 4-64 GB (~50-100 ns)
5. **SSD** — secondary storage, non-volatile, 128 GB - 4 TB (~0.1 ms = 100,000 ns)
6. **HDD** — secondary storage, non-volatile, 1-20 TB (~5-10 ms = 5,000,000 ns)

Notice how dramatic the speed differences are: RAM is roughly 1,000 times slower than cache, and an HDD is roughly 100,000 times slower than RAM. This is why the hierarchy exists — each level acts as a cache for the level below it, keeping frequently used data as close to the CPU as possible.

RAM vs ROM

Feature	RAM (Random Access Memory)	ROM (Read Only Memory)
Volatile?	**Yes** — all data lost when power is removed. This is why unsaved work is lost during a power cut.	**No** — data retained without power. Contents are permanent.
Read/Write?	Both — data can be freely read and written millions of times	Read only during normal operation (contents cannot be changed by the CPU during normal use)

Purpose	Holds the **OS**, **currently running programs**, and their **active data**. When you open a program, it is loaded from secondary storage into RAM so the CPU can access it quickly.	Stores the **BIOS/UEFI boot firmware**. When the computer is first powered on, the CPU executes instructions from ROM to perform the POST (Power-On Self-Test), check hardware, and load the OS from secondary storage into RAM.
Capacity	Typically 4-64 GB (upgradeable by adding more RAM sticks)	Typically a few MB (sufficient for boot firmware)
Speed	Fast (~50-100 ns)	Slower than RAM but still fast enough for boot-up
Effect of more	More RAM = more programs can run simultaneously without slowdown. Less reliance on slow virtual memory.	Amount of ROM is not a performance factor for the user

What happens when you open a program: The OS locates the program's files on secondary storage (HDD/SSD), copies them into RAM, and tells the CPU to begin executing the instructions at the RAM address where the program was loaded. This is why programs with more features take longer to open — more data needs to be copied from slow secondary storage into fast RAM.

Cache — Bridging the CPU-RAM Speed Gap

Cache sits between the CPU and RAM and stores copies of frequently/recently accessed data and instructions. When the CPU needs data, it checks the cache first. If the data is there (**cache hit**), it is returned very quickly. If not (**cache miss**), the CPU must fetch from slower RAM.

- **L1 cache:** Fastest, smallest, closest to CPU cores. Typically 32-64 KB per core. Accessed in ~1 ns. Often split into separate instruction and data caches.
- **L2 cache:** Larger, slightly slower. Typically 256 KB - 1 MB per core. Accessed in ~3-10 ns.
- **L3 cache:** Largest, shared between all cores. Typically 4-32 MB. Accessed in ~10-20 ns. Still much faster than RAM.

Cache works on the principle of **locality of reference**: programs tend to access the same data repeatedly (temporal locality) and access data near recently used data (spatial locality). By keeping

© RAG Learning · Not for redistribution

recently and frequently used data in cache, most CPU requests can be served without going to slower RAM.

Virtual Memory

> **Virtual memory** is a section of the hard drive/SSD used as an overflow area when RAM is full. The OS swaps (pages) data between RAM and virtual memory on demand, allowing programs that need more memory than physically available to still run. It creates the illusion of having more RAM than physically exists.

How virtual memory works (paging):

1. RAM is divided into fixed-size blocks called **pages** (typically 4 KB each). Every program's memory is split into pages.
2. The OS maintains a **page table** — a lookup table that maps each page of a program's *virtual* address space to either a physical RAM location or a location in the page file on disk.
3. When the CPU accesses a memory address, the OS checks the page table. If the required page is in RAM, the access proceeds normally.
4. If the required page is **not** in RAM (it has been swapped to disk), a **page fault** occurs. The OS pauses the program, loads the required page from the page file into RAM, updates the page table, and resumes the program. This is slow because it requires a disk access.
5. When RAM is full and a new page must be loaded, the OS identifies a page to evict (e.g., the least recently used page), **swaps it out** to the page file on disk, and loads the required page into the freed space.
6. If the evicted page is needed again later, another page fault occurs and the process repeats.

- Virtual memory is **much slower** than RAM because secondary storage access is orders of magnitude slower: an SSD page fault takes ~0.1 ms; an HDD page fault takes ~5-10 ms. RAM access takes ~50-100 ns. A single HDD page fault is roughly 100,000 times slower than a RAM access.
- Heavy use of virtual memory causes **thrashing** — the system spends more time swapping pages between RAM and disk than doing useful work, causing severe slowdown. The hard drive activity light flickers constantly and the computer becomes almost unresponsive. Every program wants a

page that another program currently has in RAM, so pages are constantly being swapped in and out.

- Virtual memory is a **temporary solution** — adding more physical RAM is the proper fix. Closing unused programs also helps, as each open program consumes RAM pages.

> **Exam Tip:** A common exam question asks: "Explain why a computer with insufficient RAM runs slowly." The answer must explain the full chain: RAM is full → OS uses virtual memory (page file on HDD/SSD) → program accesses a page not in RAM → page fault occurs → OS swaps a RAM page to disk and loads the required page from disk → disk access is ~100,000x slower than RAM → if this happens constantly, causes thrashing → system becomes very slow. The solution is to add more RAM or close programs to free RAM pages. Mentioning "page fault" and "page table" in your answer demonstrates precise knowledge and scores higher.

1.2.2 Secondary Storage

Secondary storage is non-volatile, persistent storage that retains data even when the computer is powered off. It is used for long-term storage of the OS, programs, and user files. Unlike RAM, data in secondary storage survives power loss, but it is much slower to access.

Types of Secondary Storage

Type	Technology	Speed	Capacity	Durability	Cost/GB	Best For
HDD (Hard Disk Drive)	Spinning magnetic platters with a moving read/write head. Data stored as magnetic patterns on the platter surface. The	Slow (moving parts; 5-10 ms seek time; 100-200 MB/s sequential read)	Very high (1-20 TB)	Fragile (moving parts; sensitive to shock and vibration; head crash can destroy data)	Very low	Bulk storage, backups, servers, media libraries where cost per GB matters most

	head floats nanometres above the platter and reads/writes by magnetising tiny regions.					
SSD (Solid State Drive)	NAND flash memory chips — no moving parts. Data stored as electrical charges in billions of tiny memory cells. Uses transistors to trap electrons.	Fast (0.05-0.1 ms; 500-7000 MB/s read for NVMe)	Medium-high (128 GB - 4 TB)	High (no moving parts; shock-resistant; silent)	Medium	OS drive, applications, laptops (shock resistance), gaming (fast load times)
Optical disc	Laser reads pits (indentations) and lands (flat areas) on a reflective disc surface. CD uses infrared laser; DVD uses red laser (shorter wavelength, more data); Blu-ray uses blue-violet	Slow (disc must spin to correct position)	Low (700 MB CD, 4.7 GB DVD, 25 GB Blu-ray)	Moderate (scratch-sensitive but no magnetic degradation; long archival life if stored properly)	Very low per disc	Software distribution, music/film, archival, backups

	laser (even shorter).					
Flash / USB drive	Flash memory chips in a portable casing. Same underlying technology as SSDs but smaller capacity and simpler controller.	Medium (varies widely)	Low-medium (4 GB - 1 TB)	High (solid-state, compact, no moving parts)	Low-medium	Portable file transfer, small backups, bootable media

How to Choose Storage for a Scenario

Exam questions often describe a scenario and ask you to recommend the most appropriate storage type. Consider these factors:

- **Speed:** Does the user need fast access? (SSD for OS/apps; HDD acceptable for bulk storage)
- **Capacity:** How much data needs to be stored? (HDD for large media libraries; SSD for smaller, frequently accessed data)
- **Portability:** Does the data need to be carried around? (USB flash drive; optical disc for distributing to others)
- **Durability:** Will the device be moved or subjected to shocks? (SSD or flash for laptops/portable devices; HDD is fragile)
- **Cost:** Is budget a constraint? (HDD is cheapest per GB for large amounts; optical discs are cheapest per unit)
- **Longevity:** How long does the data need to last? (Optical discs for archival; SSDs and flash drives have limited write cycles)

Magnetic Tape

Magnetic tape stores data as magnetic patterns on a thin coated plastic strip wound onto reels or inside a cartridge. Modern LTO (Linear Tape-Open) cartridges can hold up to 18 TB uncompressed. It uses **sequential access** — to reach a specific piece of data the tape must physically wind from its current position to the target location, which can take minutes.

Feature	Detail
Access type	**Sequential only** — data must be accessed in order. Cannot jump directly to any location the way an HDD or SSD can (random access). This makes it unsuitable for tasks requiring fast lookups.
Speed	Very slow for random access; fast for sequential streaming (LTO-8: ~360 MB/s sequential). Retrieving a random file can take minutes as the tape must wind to the correct position.
Capacity	Very high — LTO-9 holds 18 TB uncompressed, 45 TB compressed. A single cartridge can store an enormous amount of data.
Cost per GB	Extremely low — the cheapest storage medium per gigabyte. Ideal when cost is the dominant concern and fast access is not needed.
Durability	Excellent archival life (30–50+ years if stored correctly, away from heat, moisture, and magnetic fields). Physically fragile if bent or stretched.
Uses	Long-term offsite backups, disaster recovery archives, cloud provider cold storage (Google, AWS, Azure all use tape), broadcast archives, legal records, any data that is written once and very rarely read back.

Exam Tip: The key feature that separates magnetic tape from all other storage types is **sequential access** — you must wind through the tape to reach a specific location, unlike HDDs and SSDs which can jump directly to any sector (random access). Always state this in exam answers. Magnetic tape is the only appropriate answer for questions about *large-scale long-term archival backup at minimum cost*.

© RAG Learning · Not for redistribution

Cloud storage stores data on remote servers accessed via the internet. It is not a separate hardware type but rather uses HDDs and SSDs in data centres managed by companies. Advantages: accessible from anywhere, automatic backups, scalable, no local hardware to maintain, enables collaboration. Disadvantages: requires internet, ongoing subscription cost, data held by a third party (privacy and security concerns), speed limited by internet connection, potential data sovereignty issues (data may be stored in another country with different laws).

> **Exam Tip:** When comparing storage types, structure your answer around: speed, capacity, cost per GB, durability, portability, and whether it has moving parts. For a specific scenario, recommend the most appropriate type and justify your choice. For example: "An SSD would be best for a laptop's main drive because it is shock-resistant (no moving parts, important for a portable device), fast (improves boot time and application loading), and silent." Always give reasons specific to the scenario.

> **Exam Tip:** Key HDD vs SSD distinctions that examiners test: (1) **Fragmentation** — HDDs become fragmented over time as files are written in non-contiguous sectors; the read/write head must physically move to different locations, slowing access. **Defragmentation** reorganises files into contiguous blocks. SSDs do not suffer from fragmentation because all memory cells are accessed at equal speed (no physical movement). (2) **TRIM** — SSDs use a TRIM command to pre-erase deleted blocks in the background, maintaining write performance over time. You should NEVER defragment an SSD: it causes unnecessary writes that wear out the flash cells without any performance benefit. (3) HDDs are vulnerable to damage from physical shocks (the read/write head can crash into the platter); SSDs have no moving parts and are much more shock-resistant, making them better for laptops and portable devices.

1.2.3 Units of Data and File Size Calculations

Data Units

All data in a computer is ultimately stored as binary digits (bits). Larger units are multiples of bytes:

Unit	Size	Context

Bit	Single 0 or 1	Smallest unit of data. A single binary digit.
Nibble	4 bits	One hexadecimal digit. Can represent 0-15 (16 values).
Byte	8 bits	One character (in ASCII). Can represent 0-255 (256 values).
Kilobyte (KB)	1,000 bytes	A few paragraphs of text; a small text file.
Megabyte (MB)	1,000 KB = 1,000,000 bytes	An MP3 song (~3-5 MB); a high-resolution photo (~5-10 MB).
Gigabyte (GB)	1,000 MB	A feature-length HD film (~4-5 GB); a modern game (~30-100 GB).
Terabyte (TB)	1,000 GB	A large hard drive; a small data centre.
Petabyte (PB)	1,000 TB	Large data centre scale; all of Netflix's video content is ~tens of PB.

SI (decimal) vs binary prefixes: In computing, two different definitions of "kilobyte" exist. The **SI (International System) definition** uses powers of 10: 1 KB = 1,000 bytes, 1 MB = 1,000,000 bytes, 1 GB = 1,000,000,000 bytes. The **binary definition** uses powers of 2: 1 KiB (kibibyte) = 1,024 bytes, 1 MiB = 1,048,576 bytes, 1 GiB = 1,073,741,824 bytes. Hard drive manufacturers use SI (makes drives look bigger); operating systems historically used binary (which is why a "500 GB" drive shows as ~465 GB in Windows).

Unit (SI/decimal)	Exact bytes (SI)	Binary equivalent	Exact bytes (binary)
1 KB (kilobyte)	1,000 bytes	1 KiB (kibibyte)	1,024 bytes
1 MB (megabyte)	1,000,000 bytes	1 MiB (mebibyte)	1,048,576 bytes
1 GB (gigabyte)	1,000,000,000 bytes	1 GiB (gibibyte)	1,073,741,824 bytes
1 TB (terabyte)	1,000,000,000,000 bytes	1 TiB (tebibyte)	1,099,511,627,776 bytes

© RAG Learning · Not for redistribution

Note: OCR uses decimal prefixes (1 KB = 1,000 bytes) in exam calculations. In exam questions, always use 1,000 unless the question specifies otherwise. The gap between SI and binary grows at larger scales: at the GB level the difference is about 7%, at the TB level it is about 10%.

File Size Formulae

Sound file size (uncompressed):

```
File size (bits) = sample rate (Hz) x duration (seconds) x bit depth (bits) x channels
File size (bytes) = file size (bits) / 8

Example 1: Short stereo clip (common exam format)
  44,100 Hz sample rate, 10 seconds, 16-bit depth, 2 channels (stereo)
  = 44,100 x 16 x 10 x 2 = 14,112,000 bits
  = 14,112,000 / 8 = 1,764,000 bytes
  = 1,764,000 / 1,000,000 = 1.764 MB uncompressed
  (An MP3 version of the same clip would typically be ~0.16 MB at 128 kbps
   — about 11x smaller, illustrating the impact of lossy compression)

Example 2: Mono recording
  44,100 Hz sample rate, 60 seconds, 16-bit depth, 1 channel (mono)
  = 44,100 x 60 x 16 x 1 = 42,336,000 bits
  = 42,336,000 / 8 = 5,292,000 bytes
  = 5,292,000 / 1,000,000 = 5.29 MB

Example 3: Stereo recording (same settings as Example 2 but 2 channels)
  = 44,100 x 60 x 16 x 2 = 84,672,000 bits
  = 84,672,000 / 8 = 10,584,000 bytes = 10.58 MB
  (Stereo is exactly double mono because there are 2 channels)

Example 4: How long can you record on 100 MB at CD quality (stereo)?
  100 MB = 100,000,000 bytes = 800,000,000 bits
  Duration = 800,000,000 / (44,100 x 16 x 2)
  Duration = 800,000,000 / 1,411,200
  Duration = 566.9 seconds ≈ 9 minutes 27 seconds
```

Image file size (uncompressed bitmap):

```
File size (bits) = width (pixels) x height (pixels) x colour depth (bits per pixel)
File size (bytes) = file size (bits) / 8
```

```
Example 1: Full HD image
  1920 x 1080 pixels, 24-bit colour depth (true colour)
  = 1920 x 1080 x 24 = 49,766,400 bits
  = 49,766,400 / 8 = 6,220,800 bytes = 6.22 MB

Example 2: How many 4-megapixel photos fit on a 2 GB memory card?
  4 megapixels = 4,000,000 pixels, assume 24-bit colour
  Each photo = 4,000,000 x 24 / 8 = 12,000,000 bytes = 12 MB
  2 GB = 2,000 MB
  Number of photos = 2,000 / 12 = 166 photos (approximately)

Example 3: What colour depth allows a 640 x 480 image in 150 KB?
  150 KB = 150,000 bytes = 1,200,000 bits
  Colour depth = 1,200,000 / (640 x 480) = 1,200,000 / 307,200 = 3.9
  So a colour depth of 3 bits (8 colours) would fit; 4 bits (16 colours) would not.
```

Text file size:

```
File size (bits) = number of characters x bits per character
File size (bytes) = file size (bits) / 8

Example: 2000 characters, ASCII (8 bits per character)
  = 2000 x 8 = 16,000 bits = 2,000 bytes = 2 KB

Example: 5000 characters, Unicode UTF-16 (16 bits per character)
  = 5000 x 16 = 80,000 bits = 10,000 bytes = 10 KB
```

> **Exam Tip:** Always show your working clearly in calculation questions — partial marks are awarded for correct method even if the final answer is wrong. Remember to divide by 8 to convert bits to bytes. For sound, always check whether it is mono or stereo (stereo = multiply by 2 for two channels). These are common multi-mark questions that students lose marks on by rushing or forgetting a step.

1.2.4 Number Representation

Computers use binary (base 2) internally, but humans find denary (base 10) and hexadecimal (base 16) more convenient. You must be able to convert between all three and perform binary arithmetic.

Number Systems Overview

System	Base	Digits Used	Where Used
Binary	2	0, 1	How computers store and process all data internally
Denary	10	0-9	Human everyday counting system
Hexadecimal	16	0-9, A-F	Compact representation of binary; colour codes (#FF0000); MAC addresses; memory addresses; error codes

Why hexadecimal? Binary numbers are long and error-prone for humans. Hexadecimal provides a compact representation: each hex digit maps to exactly 4 binary digits (one nibble). So an 8-bit binary number becomes just 2 hex digits, and a 32-bit address becomes 8 hex digits. This makes hex much easier to read and less prone to transcription errors.

Conversion Methods

Conversion	Method	Worked Example
Denary → Binary	Write place values (128, 64, 32, 16, 8, 4, 2, 1). Subtract the largest that fits, write 1 for that position, 0 otherwise.	75: 75-64=11, 11-8=3, 3-2=1, 1-1=0. Bits at 64,8,2,1 = `01001011`
Binary → Denary	Multiply each bit by its place value (128, 64, 32, 16, 8, 4, 2, 1) and sum all results.	`01001011` = 0+64+0+0+8+0+2+1 = 75
Denary → Hex	Divide by 16: quotient = left digit, remainder = right digit. Remainders 10-15 use A-F.	75 / 16 = 4 remainder 11 (B) → `4B`
Hex → Denary	Multiply each digit by its power of 16 and sum.	`4B` = (4 x 16) + (11 x 1) = 64 + 11 = 75
Binary → Hex	Split into nibbles (groups of 4 from right); convert each nibble to its hex digit.	`0100 1011` → 4 B → `4B`

Hex → Binary	Convert each hex digit to its 4-bit binary equivalent.	`4B` → 0100 1011 → `01001011`

Binary Addition

Binary addition follows four rules, just like denary addition but with only two digits:

```
Rules: 0+0 = 0    0+1 = 1    1+0 = 1    1+1 = 10 (write 0, carry 1)
       1+1+1 = 11 (write 1, carry 1)

Example: 01001011 (75) + 00110010 (50)
  Carry:  1 1 1
          0 1 0 0 1 0 1 1    (75)
        + 0 0 1 1 0 0 1 0    (50)
        -------------------
          0 1 1 1 1 1 0 1    (125)

  Check: 64+32+16+8+4+1 = 125. Correct! (75 + 50 = 125)

Overflow example: 11001000 (200) + 01100100 (100)
  The result is 300, but 300 > 255 (maximum for 8 bits).
  An OVERFLOW ERROR occurs — the result cannot be stored in 8 bits.
  The 9th bit (carry out) is lost, giving a wrong answer.
```

Binary Shifts

Binary shifts move all bits left or right by a specified number of positions. They are a very fast way to multiply or divide by powers of 2:

Shift	Operation	Effect	Example
Left shift by n	Move all bits left by n positions; fill right with zeros	Multiplies by 2^n	`00000011` (3) << 2 = `00001100` (12). 3 x 2^2 = 3 x 4 = 12.
Right shift by n	Move all bits right by n positions; fill left with zeros	Integer divides by 2^n (rounds down)	`00001100` (12) >> 1 = `00000110` (6). 12 / 2^1 = 6.

Important: bits shifted off the edge are lost permanently. A left shift can cause overflow if a 1 is shifted out of the most significant position. A right shift of an odd number loses the remainder: `00000111` (7) >> 1 = `00000011` (3), not 3.5 — the 0.5 is lost.

```
More shift examples:
  00000101 (5) << 1 = 00001010 (10)    — 5 x 2 = 10
  00000101 (5) << 3 = 00101000 (40)    — 5 x 8 = 40
  00110000 (48) >> 2 = 00001100 (12)   — 48 / 4 = 12
  00001001 (9) >> 1 = 00000100 (4)     — 9 / 2 = 4 (remainder lost)
```

1.2.5 Characters, Images and Sound

Character Sets

A **character set** is a mapping between characters (letters, digits, symbols) and binary numbers. Every character stored in a computer has a unique binary code.

Feature	ASCII	Unicode
Bits per character	7 bits (128 characters) or 8 bits extended ASCII (256 characters)	Variable: UTF-8 uses 1-4 bytes; UTF-16 uses 2-4 bytes; UTF-32 uses 4 bytes
Characters supported	English letters (A-Z, a-z), digits (0-9), punctuation, control characters (newline, tab)	All world scripts: Latin, Cyrillic, Arabic, Chinese, Japanese, Korean, emoji, mathematical symbols, historic scripts — over 149,000 characters
File size impact	Smaller (1 byte per character for English text)	Larger (1-4 bytes per character depending on encoding and script)
Key facts	A=65, a=97, 0=48. Uppercase and lowercase differ by 32. Characters are sequential (B=66, C=67, etc.). Space = 32.	UTF-8 is backward compatible with ASCII — the first 128 characters are identical. UTF-8 is the most common encoding on the web.

Why was Unicode needed? ASCII only supports 128 (or 256) characters, which is sufficient for English but cannot represent Chinese characters (tens of thousands), Arabic, Hindi, Japanese, or emoji. As computing became global, a universal character set was needed. Unicode solves this by assigning a unique code point to every character in every writing system.

Images

Digital images are represented as a grid of tiny coloured squares called **pixels** (picture elements). Each pixel stores a binary value representing its colour.

- **Colour depth** = the number of bits used to represent the colour of each pixel. More bits = more possible colours. 1 bit = 2 colours (black/white); 8 bits = 256 colours; 24 bits = 16.7 million colours (true colour, using 8 bits each for red, green, and blue).
- **Resolution** = the total number of pixels in the image (width x height). Higher resolution = more detail and sharper image = larger file size. A 1920 x 1080 image has 2,073,600 pixels (about 2 megapixels).
- **Metadata** stored in the file header includes: width, height, colour depth, file format, and potentially camera settings (EXIF data).
- Increasing colour depth improves colour accuracy but increases file size. Increasing resolution improves detail but also increases file size. There is always a trade-off between quality and file size.

Sound

Sound in the real world is an analogue wave — a continuously varying signal. Computers are digital and can only store discrete values, so analogue sound must be converted to digital through a process called **sampling**.

- **Sampling** means measuring the amplitude (height) of the sound wave at regular time intervals and recording each measurement as a binary number.
- **Sample rate** = the number of samples taken per second, measured in Hertz (Hz). CD quality = 44,100 Hz (44,100 measurements per second). A higher sample rate captures more detail of the original wave, including higher frequencies.
- **Bit depth** = the number of bits used to store each sample. More bits = more precise amplitude levels = less **quantisation error** (the difference between the actual amplitude and the nearest digital level). CD quality = 16 bits (65,536 possible levels).

© RAG Learning · Not for redistribution

- **Nyquist theorem:** the sample rate must be at least **twice** the highest frequency in the sound to accurately reproduce it. Human hearing goes up to ~20,000 Hz, so CD audio samples at 44,100 Hz (just over double).
- Higher sample rate AND higher bit depth both increase quality AND file size. This is the fundamental trade-off in digital audio.

1.2.6 Compression

Compression reduces the file size of data so it takes up less storage space and can be transmitted faster over networks. There are two fundamentally different approaches:

Feature	Lossy Compression	Lossless Compression
Data loss	Yes — some data is permanently removed (data deemed imperceptible to humans)	No — original data is fully recoverable after decompression. Not a single bit is lost.
File size reduction	Much smaller (high compression ratios, e.g., 10:1 or more)	Moderately smaller (typical 2:1 to 3:1 ratio)
Quality	Reduced (may be imperceptible at high quality settings, but degrades with more compression)	Identical to original — no quality loss whatsoever
Reversible?	No — cannot recover original from compressed file. Lost data is gone forever.	Yes — decompression gives the exact original file, bit for bit
Formats	JPEG (images), MP3/AAC (audio), MP4/H.264/H.265 (video)	PNG (images), FLAC/WAV (audio), ZIP/7z (any files), GIF (images, limited colours)
Best for	Media where slight quality loss is acceptable: streaming video, web images, music for portable players, social media	Data where accuracy is critical: text documents, executables, source code, medical images, archival, financial records

© RAG Learning · Not for redistribution

How it works	Removes data humans cannot easily perceive: inaudible frequencies in audio, imperceptible colour differences in images, redundant information between video frames	Finds patterns and encodes them more efficiently without removing any data. Techniques include: **Huffman coding** (frequent characters get shorter codes), **RLE** (Run Length Encoding — "AAAA" becomes "4A"), **dictionary compression** (LZW — repeated sequences replaced with references)

Run-Length Encoding (RLE) — Worked Example

RLE is a simple lossless technique that replaces consecutive repeated values with a count and a value. It is particularly effective for images with large areas of the same colour (e.g., a blue sky, a white background).

```
Original pixel row (colour values): R R R R R B B B B G G R R
Encoded with RLE:                   (R,5) (B,4) (G,2) (R,2)

Original: 13 values stored
Encoded:  4 pairs stored (8 values) — saves 38% of storage

Less effective case (no repeated values):
Original: R B G R B G R B G R B G
Encoded:  (R,1)(B,1)(G,1)(R,1)(B,1)(G,1)(R,1)(B,1)(G,1)(R,1)(B,1)(G,1)
Result:   Encoded is LARGER than original! RLE works poorly on
          varied data — this is why complex photographs use JPEG
          rather than purely RLE-based compression.

Real-world use: RLE is used within fax transmission and BMP files
for simple graphics. More advanced lossless codecs (ZIP, PNG) use
RLE in combination with other techniques for better results.
```

Exam Tip: Run-length encoding is only effective when there are long runs of repeated data. For text or complex images with many different colours, RLE may actually **increase** the file size. This is why JPEG (lossy) is used for photographs and PNG (lossless, using a more sophisticated algorithm) is used for logos and diagrams rather than raw RLE. If an exam question asks you to apply RLE, write out each run as (value, count) or (count, value) — both are accepted; be consistent.

© RAG Learning · Not for redistribution

When to Use Each Type

```
Scenario: Storing holiday photos to share on social media
  → Lossy (JPEG). Slight quality loss is imperceptible on screens,
    and much smaller files upload faster and use less data.

Scenario: Archiving a medical X-ray for a hospital database
  → Lossless (PNG). Any data loss could obscure diagnostic detail,
    potentially leading to misdiagnosis. Accuracy is critical.

Scenario: Sending a software installer via email
  → Lossless (ZIP). Losing even one bit of an executable file would
    corrupt it and prevent it from running. Software must be exact.

Scenario: Streaming music on a mobile phone with limited data
  → Lossy (MP3/AAC). Compression ratios of 10:1 make streaming
    practical. Most listeners cannot tell the difference at 256 kbps.

Scenario: Backing up important text documents
  → Lossless (ZIP). Text files are typically small anyway, and
    no data can be lost from important documents.
```

Exam Tip: Know the key difference: lossy = quality lost permanently (cannot undo); lossless = fully recoverable (no quality loss). For a given scenario, justify your choice clearly. File size calculation formulae give the **uncompressed** size — actual files are usually compressed and smaller than the calculated value. A common mistake is saying "lossy compression loses some of the file" without specifying that it removes data *humans cannot easily perceive* — this detail matters for full marks.